

de modo que nenhum vértice tenha a mesma cor que um adjacente

Coloração $\rightarrow$ atribuição de uma cor (ou label) p/ cada vértice. N° cromático = ^{mínimo de cores} necessário p/ colorir

Algoritmo guloso: ordena vértices em ordem decrescente de grau e, para cada um deles, atribui a menor cor com a qual não tem adjacência

Alteração Estrutural $\rightarrow$ transforma o grafo em um grafo completo K_m . O menor m dos grafos gerados é o n° cromático. P/ cada grafo usa a alteração α (acrescenta aresta entre 2 vértices não adjacentes) e β (condemna os 2 vértices, tornando-os arestas de ambos).

Rede $\rightarrow$ possui um vértice S com apenas arestas de saída e um T com apenas arestas de entrada. Tirando esses 2, entrada deve ser igual a saída. Fluxo da rede é tudo que sai da origem e capacidade é tudo que chega em T .

Ford-Fulkerson $\rightarrow$ calcula o fluxo máximo em uma rede

$\rightarrow$ cria uma rede residual, com arestas que invertem os sentidos:

aresta direta: quando capacidade $>$ fluxo, ainda há fluxo disponível de V p/ U . O valor da aresta é a diferença entre capacidade e fluxo

aresta contrária: quando fluxo $>$ 0, pode retornar (desfazer) o fluxo previamente alocado, gerando aresta de U p/ V com valor igual ao fluxo

Se há um caminho de S a T na rede residual, pode aumentar o fluxo de todo o caminho na rede original em X (sendo X o ^{menor} valor no ^{caminho}).
Gera uma nova rede residual e repete até não houver mais caminhos de S a T .

Produtor X Consumidor $\rightarrow$ vértices que recebem de S são produtores (produzindo o valor da aresta (S, V)) e os que incidem em T são consumidores (consumindo o valor da aresta (V, T)). Demais vértices e arestas fazem parte do caminho de transporte que pode limitar o transporte. Quando um vértice possui restrição divide em 2 vértices V' e V'' , com uma aresta (V', V'') com valor ^{igual à} restrição.
Usa Ford-Fulkerson depois

«cobertura de arestas» conjunto de arestas adjacente a todos os vértices de G

1) de vértices $\rightarrow$ conjunto de vértices adjacente a todas as arestas de G

vértice dominante $\rightarrow$ 1) de vért. em que todo vértice de G é adjacente a algum vértice do conj. ou faz parte do próprio conj.

Associação Máxima $\rightarrow$ empacotamento dos vértices, formando uma rede e usando Ford-Fulkerson se encontra a associação máxima. Preflight-Karp também funciona, associando um vértice de um lado com outro do outro até todos terem um par. Em grafos não bipartidos um caminho alterna

PERT-CPM $\rightarrow$ vértices representam a conclusão de tarefas, orientar as tarefas (valor da tarefa e a duração). Caminho crítico são as atividades obrigatórias de terminar em tempo p/ não atrasar o projeto.

data mais cedo $\rightarrow$ momento mais cedo que pode ser concluída a atividade. A data mais cedo da última atividade é a duração do projeto. Data mais tarde começa do fim e vai p/ o começo.

Quando se recebe de duas fontes: DMC $\rightarrow$ Maior / DMT $\rightarrow$ Menor. Caminho crítico são os vértices que DMC = DMT

Ordenação Topológica $\rightarrow$ ordena o grafo deixando predecessores antes dos antecessores. Remova os vértices do grau 0 e seus arestas e o adiciona ao caminho, ou DFS que adiciona ao caminho na volta da recursão (resulta invertido)

Componentes Fortemente Conexas $\rightarrow$ pode chegar em qualquer vértice a partir de qualquer vértice. Kosaraju: DFS por ordem para numerar os vértices. Gera G' com arestas invertidas e realiza DFS em G' começando pelo último a ser adicionado no caminho na 1ª DFS.

Cada DFS resulta em uma componente fortemente conexa (repete a 2ª DFS até todos vértices estarem parte de 1 componente)

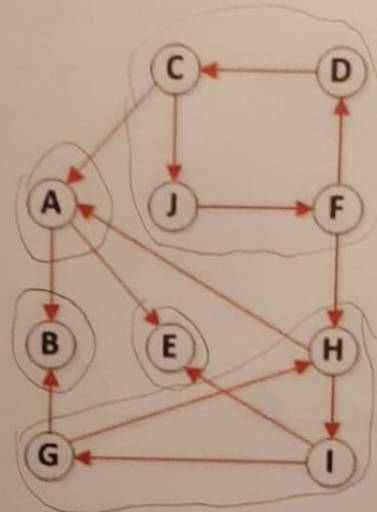
Caminho Viajante $\rightarrow$ programação dinâmica. Ganga recursivamente ponto de partida e todos que devem ser visitados, salvando os caminhos e mantendo o menor.

$$F(S, \{a, \dots, T\}) \rightarrow F(a, \{b, \dots, T\}) + G_{S,a} \dots F(T, \{ \}) + G_{a,T} \rightarrow G_{T,S}$$

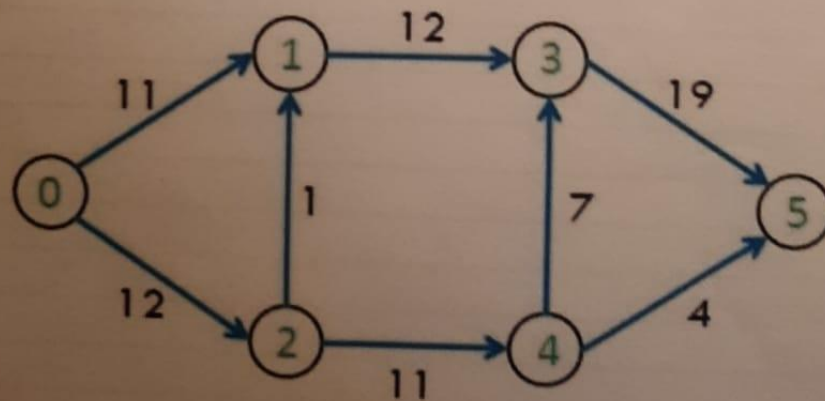
$\rightarrow$ em casos com dados excludentes, modela o grafo apenas com os arestas que geram a exceção

1) (2.0 pontos) Faça uma função que receba uma matriz de adjacências $G[10][10]$ de um grafo não dirigido e verifique se o conjunto de arestas nele contido representa uma associação perfeita, ou seja, um conjunto de arestas independentes que cobre todos os vértices.

2) (1.5 pontos) Identifique as componentes fortemente conexas do grafo a seguir. :



3) (1.5 pontos) Considere a rede abaixo, em que o número em cada aresta representa sua capacidade. A partir de um fluxo zerado, mostre a rede residual nas duas primeiras iterações do algoritmo de Ford Fulkerson de cálculo de fluxo máximo:



5.0

```
1) int assoc_perf(int G[][10]) {  
    int parceiros[10];  
    for (int i=0; i<10; i++)  
        parceiros[i] = 0;
```

2.0

```
    for (int i=0; i<10; i++)  
        for (int j=0; j<10; j++)  
            if (G[i][j] != -1)
```

```
                parceiros[i] += 1;
```

// como é não dirigido, o grau sempre
// deve ser igual a 1, caso contrário,
// maior de um part. de cada um de um b

```
    int resultado = 1;
```

```
    for (int i=0; i<10; i++)
```

```
        if (parceiros[i] != 1)
```

// se for 0, significa que nenhuma vez
// apareceu o nó

```
            resultado = 0;
```

// se for > 1, significa que o vértice possui os

```
            break;
```

// adjacentes e, logo, não é independente

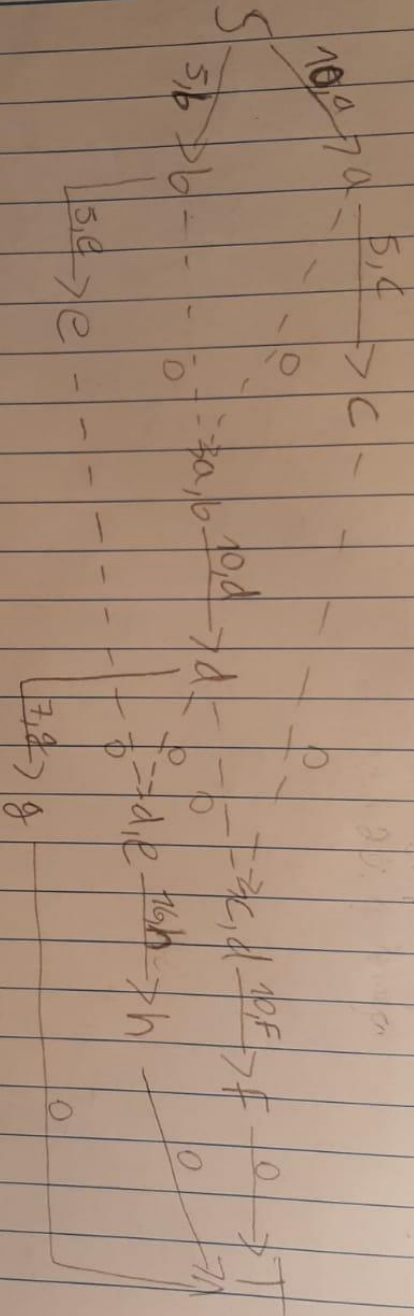
```
    return resultado;
```

// 0 -> não é uma associação perfeita
// 1 -> é uma associação perfeita

```
}
```


nutrientes = concentração da força
deixa = mais - mais - mais - mais

1.5

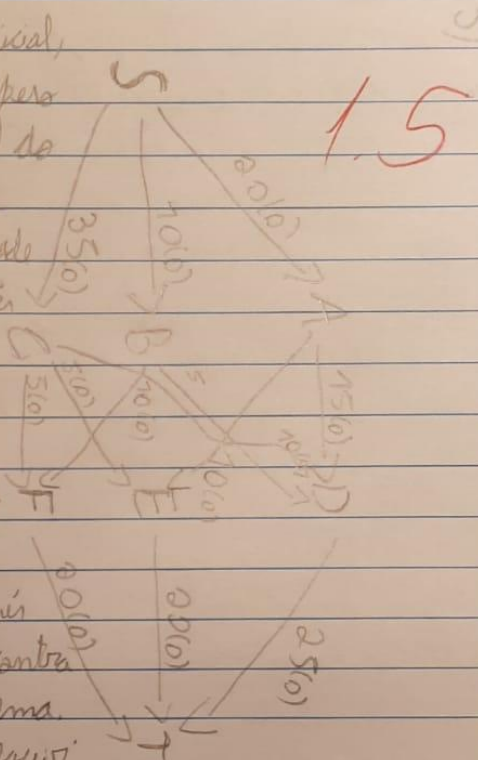


	s	a	b	c	a/b	d	e	gd	f	g	d/g	h	j	
0	10	5	15	10	20	10	20	30	30	27	20	36	36	data-main-std
1	10	10	36	10	20	20	20	36	36	36	20	36	36	data-main-std
2	0	3	11	0	0	10	6	6	6	9	0	0	0	data-main-std

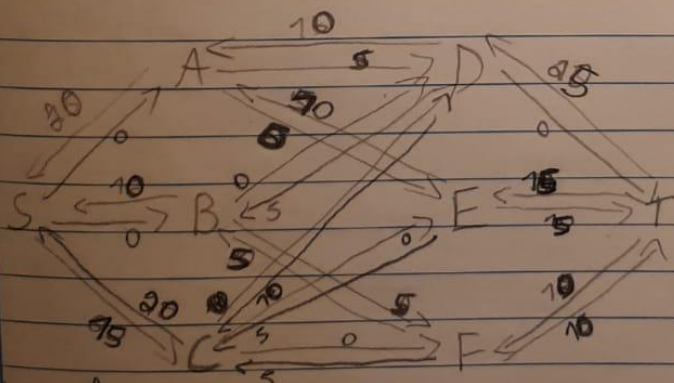
Tempo mínimo: 36

Comintha citratis: a, d, h

- Gera-se uma rede com um vértice inicial, que possui arestas para A, B e C, com peso de aresta igual a quantidade disponível do produto. A, B, e C terão arestas para D, E e F com peso igual a disponibilidade de transporte, respeitando as informações na tabela. D, E e F possuem arestas para um vértice final, com peso igual a quantidade de produto requisitada. O grafo gerado é este ao lado:

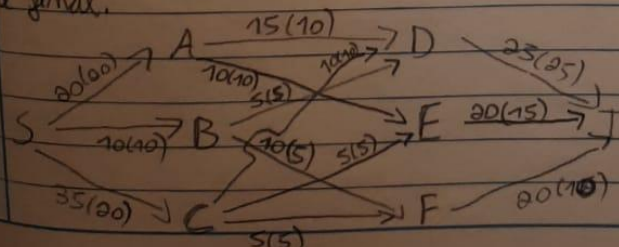


Depois, gera a rede residual e, através do algoritmo Ford-Fulkerson, se encontra a melhor resolução para o problema. A rede residual final é esta a seguir:



- S-A-D-T → 15
- S-A-E-T → 5
- S-B-D-T → 5
- S-B-F-T → 5
- S-C-D-T → 5
- S-C-E-T → 5
- S-C-F-T → 5
- S-C-D-A-E-T → 5

Rede final:



conclusão no texto →

→ no caso dessa situação, não é possível nem transportar tudo
que é produzido e nem suprir toda a demanda.

Universidade de Caxias do Sul
Área de Ciências Exatas e da Tecnologia
Teoria de Grafos – Prof. Ricardo Dorneles
Área 2 - Parte 2 – 03/07/23

1) (2.0 pontos) Um conjunto dominante é um subconjunto de vértices tal que todo vértice do grafo está no conjunto ou é adjacente a um dos vértices. Escreva uma função (pode ser pseudo-código) que receba uma matriz de adjacências $G[10][10]$ e um vetor $Dom[10]$ representando um subconjunto de vértices de G (os vértices pertencentes ao subconjunto estão marcados com 1 no vetor, os que não pertencem estão marcados com 0) e verifique se o subconjunto descrito em $Dom[10]$ é um Conjunto Dominante de Vértices, retornando: 1 - Se Dom é um conjunto dominante; 0 - Se Dom não é um conjunto dominante.

2) (1.5 pontos) Construa um diagrama PERT da tabela de tarefas a seguir. Calcule também o tempo mínimo para completá-lo, as ATIVIDADES do caminho crítico e, para cada evento, a data mais cedo, a data mais tarde e a folga.

Tarefa	Predecessoras	Duração
a	Nenhum	10
b	Nenhum	5
c	a	5
d	a,b	10
e	b	5
f	c,d	10
g	d	7
h	d,e	16

3) (1.5 pontos) De três depósitos A, B e C, dispondo respectivamente de 20, 10 e 35 toneladas de um dado produto, pretende-se fazer chegar a três destinos D, E e F, respectivamente 25, 20 e 20 toneladas do produto. As disponibilidades de transporte em caminhão entre os diferentes pontos são os seguintes:

	D	E	F
A	15	10	-
B	5	-	10
C	10	5	5

Mostre como a teoria de grafos pode ser utilizada para estabelecer o melhor plano de transportes, e encontre a solução para a situação específica descrita nessa questão.

* Não dirigido

5.0

```
1) int dominante(int G[10][10], int dom[10]) {  
    int conq[10] = {0};
```

```
    for (int i=0; i<10; i++) {  
        if (dom[i] == 1) {  
            conq[i] = 1;  
            for (int j=0; j<10; j++) {  
                if (G[i][j] == 1) {  
                    conq[j] = 1;  
                }  
            }  
        }  
    }
```

2.0

```
    for (int i=0; i<10; i++) {  
        if (conq[i] == 0) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

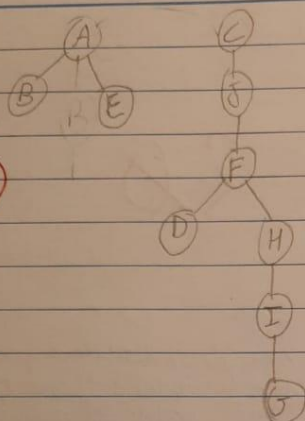
	1	2	3	4	5	6	7	8	9	10		
Dom =	{	0	0	1	0	1	0	0	1	0	0	}
conq =	{	1	1	1	1	1	1	1	1	1	1	}

1 / 1

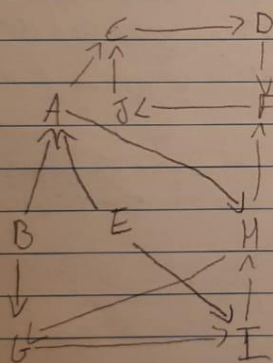
B-E-A-D-G-I-H-F-J

2)

1.5



G:



C-D-F-J

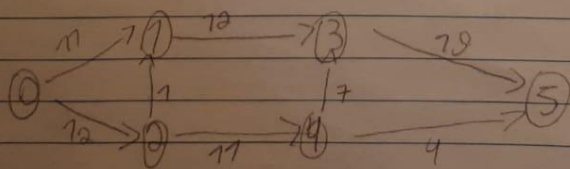
H-G-I

A

E

B

3) G' - 1ª iteração:



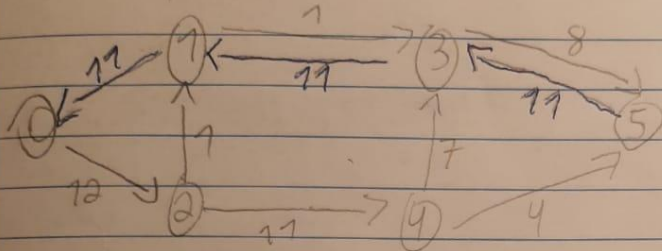
0-1-3-5 → aumenta 12

1.5

constrói aresta contrária (fluxo > 0)
 lápis: aresta direta (capacidade - fluxo > 0)



G' - iteração 2:



caminho 1
0-1-3-5
caminho 2
0-2-4-5
caminho 3
0-2-5

G' - iteração 3:

